

Object-oriented design principles (Advanced Topics)

Software Engineering (DE52)

15 avril 2025 – Yazan MUALLA

CIAD, UTBM, France

Table of contents

- Information Hiding
- Coupling and Cohesion
- Separation of Concerns
- Conceptual Integrity
- Inheritance Issues
- Modular design: Example
- Model Checking

Information Hiding

A series of horizontal lines in various shades of green and white, extending from the left edge of the slide towards the right, positioned below the title.

Information Hiding

- The module should only have access to the information that it needs to do its job
- Information hiding allows models of the system to give others the minimum amount of information needed to use them correctly and hide everything else
- In general, things that might change, like implementation details, should be hidden. And things that should not change, like assumptions, are revealed through interfaces
- Information hiding is often associated with encapsulation. Obviously!

Information Hiding: Example

- The **String** library has a method called *concat*
- As a user of the library you are given access to the concatenation functionality through an interface, a particular method signature
- You are not shown how the functionality is implemented. That is because the implementation may change and you should not depend on how it actually works

```
String a = "Hello";  
String b = a.concat("World");
```

Information Hiding: Access modifiers

- Attributes can also be hidden to prevent critical information of a classroom being changed directly
- Access modifiers change which classes can access attributes and behaviors. They also determine which attribute and behaviors a superclass will share with its subclasses
- A reminder: There are four levels of access in Java:
 - public
 - protected
 - default
 - private

Information Hiding: Access modifiers

- Attributes with a **public** access modifier are accessible by any class in your system. But this access does not allow other classes to change the implementation of the behavior for the method

```
public class Person {  
    public String name;  
  
    public void sleep(int n) {  
        wait(n);  
    }  
}  
  
public static void main(String[] args) {  
    Person ym = new Person();  
    ym.name = "Uchiha Itachi";  
    ym.sleep(6000);  
}
```

Information Hiding: Access modifiers

- **Protected** behaviors and attributes are only available to the encapsulating class itself, all subclasses, and classes within the same package
- **Packages** are the means by which Java organizes related classes into a single namespace

```
package Informatique;  
public class Person {  
    protected String name;  
  
    public Person(String newName) {  
        this.name = newname;  
    }  
    public String getName() {  
        return this.name;  
    }  
}}
```

Information Hiding: Access modifiers

```
package Informatique;  
public class Person {  
    protected String name;  
    public Person(String newName) {  
        this.name = newname; }  
    public String getName() {  
        return this.name;  
    }  
}
```

```
package Telecommunication;  
import Informatique.*;  
public class University {  
    private String name = "UT";  
    private List<Person> students = new ArrayList<Person>();  
  
    public static void main(String[] args) {  
        University uni = new University();  
        Person student = new Person("Lelouch Lamperouge");  
        Student.name = "Eren Yeager";  
        uni.students.add(student);  
    }  
}
```

Information Hiding: Access modifiers

- The **default** access modifier will only allow access to attributes and methods in the **package** and the **encapsulating class**. This access modifier is also called the **no modifier access** because you do not need to explicitly declare it
- **Private** attributes and methods are not accessible by any class other than by the encapsulating class itself. This means these attributes cannot be accessed directly and these methods cannot be invoked by any other classes

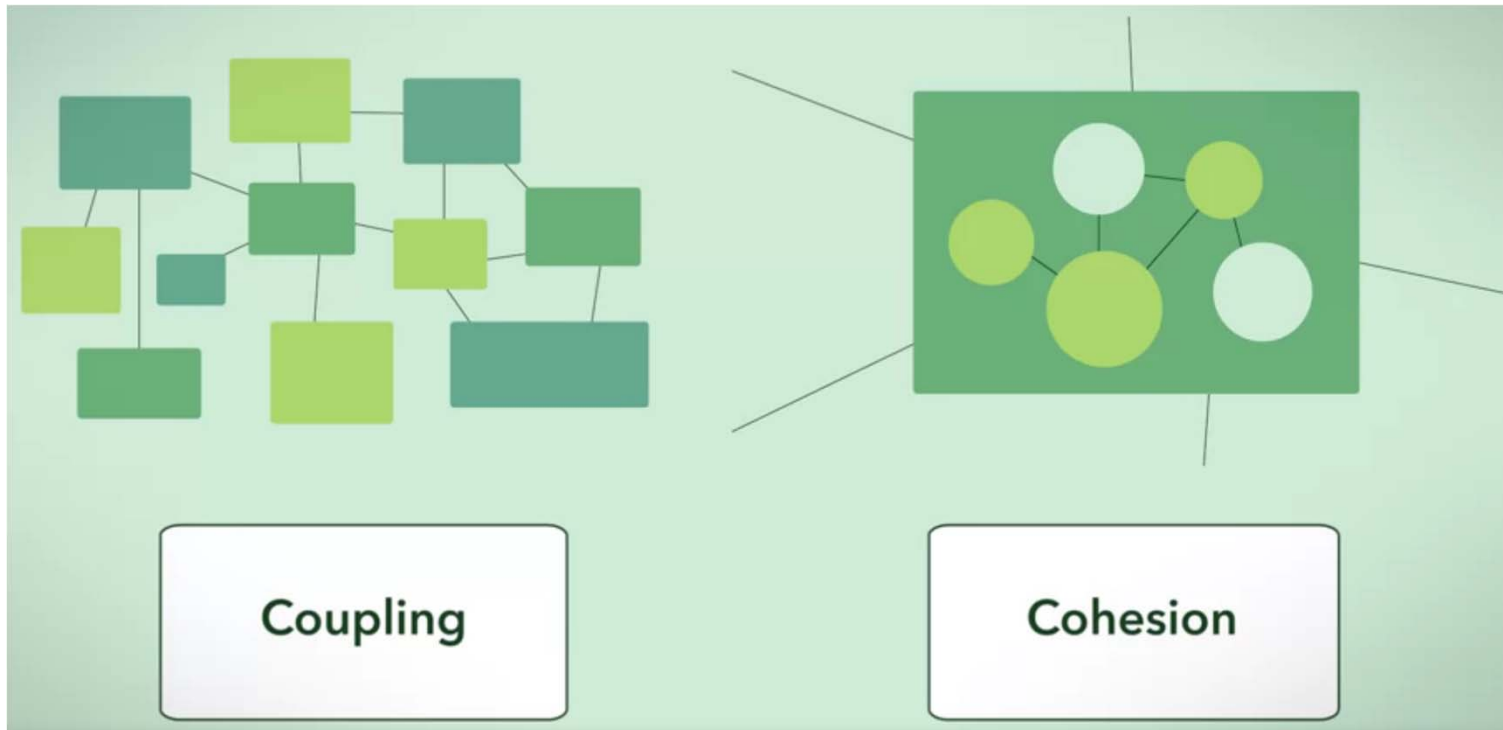
Coupling and Cohesion

A decorative graphic consisting of several horizontal lines in shades of green and white, located at the bottom of the slide.

Design complexity

- The average person can only hold **seven** things in short term memory. This was observed in a psychology paper by George Miller in which subjects had to recall 1 to 14 random sounds and images
- When you are programming, keeping modules simple is critical. Once you design complexity exceeds what developers can mentally handle, bugs will occur more often
- We need a way of evaluating your design complexity

Design complexity



Complexity between a
module and other modules

Complexity within a module

Coupling



Tightly coupled vs. Loosely coupled

- Coupling for a module captures the complexity of connecting the module to other modules
- If your module is highly reliant on other modules, you would say this module is **tightly coupled** to others. This is like having puzzle pieces
- On the other hand, if your module finds it easy to connect to other modules, this module is **loosely coupled** to others. This is like Lego
- You want coupling for your module to be loose or low, not tight

Evaluating the coupling of a module

- **Degree** is the number of connections between the module and others. With coupling, you want to keep the degree small. For instance, if the module needed to connect to other modules through a few parameters or interfaces, then degree would be small and coupling would be loose
- **Ease** is how obvious are the connections between the module and others. With coupling, you want the connections to be easy to make without needing to understand the implementations of the other modules
- **Flexibility** is how interchangeable the other modules are for this module. With coupling, you want the other modules easily replaceable for something better in the future

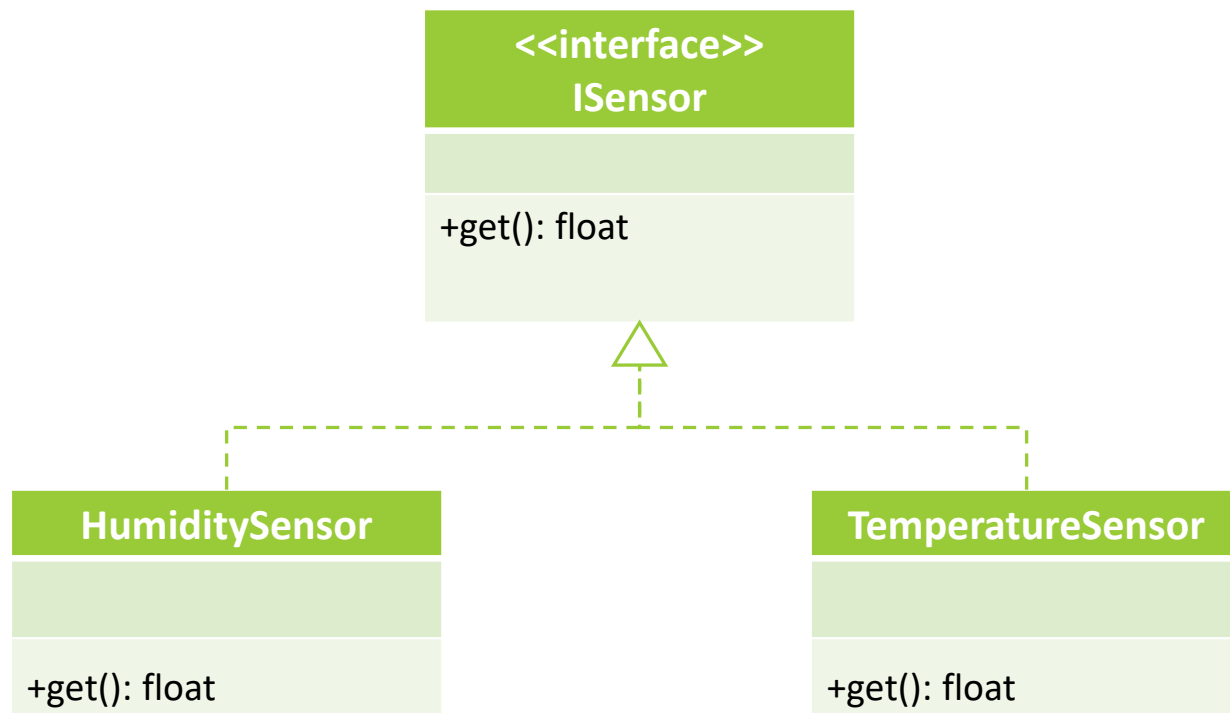
High cohesion vs. Low cohesion

- Cohesion represents the clarity of the responsibilities of a module
- If your module performs one task and nothing else or has a clear purpose, your module has high cohesion
- If your module tries to encapsulate more than one purpose or has an unclear purpose, your module has low cohesion
- You want high cohesion. If you find your module having more than one responsibility, it is probably time to split your module

Cohesion and coupling in practice

```
public float get (int controlFlag) {  
    switch (controlFlag) {  
        case 0:  
            return this.humidity;  
            break;  
        case 1:  
            return this.temperature;  
            break;  
        default:  
            throw new  
UnknownControlFlagException();  
    }  
}
```

Low coupling and high cohesion



Highly cohesive and Loosely coupled

Low coupling and high cohesion (a balance!)

- For a complex system, the complexity can be distributed **between** the modules or **within** the modules
- As modules are simplified to achieve high cohesion, they may need to depend more on other modules thus increasing coupling
- As connections between modules are simplified to achieve low coupling, the modules may need to take on more responsibilities thus lowering cohesion
- you're going to need to start applying good design to your system so that they have **low coupling and high cohesion**

Separation of Concerns

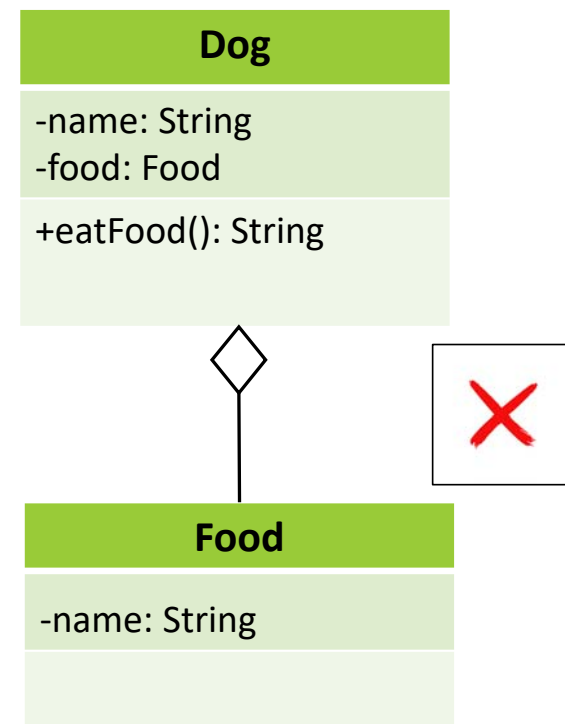
A decorative graphic consisting of several horizontal lines in shades of green and white, located at the bottom of the slide.

Separation of Concerns

- A concern is a very general notion, basically it is anything that matters in providing a solution to a problem
- Example: The concerns in a supermarket could be:
 - How to bake bread?
 - How to organize products in the shelves?
 - how to accept payment?
- There are separate departments that focus on each concern. Each concern poses unique sub-problems, and each department knows what to do and how to address their specific concerns

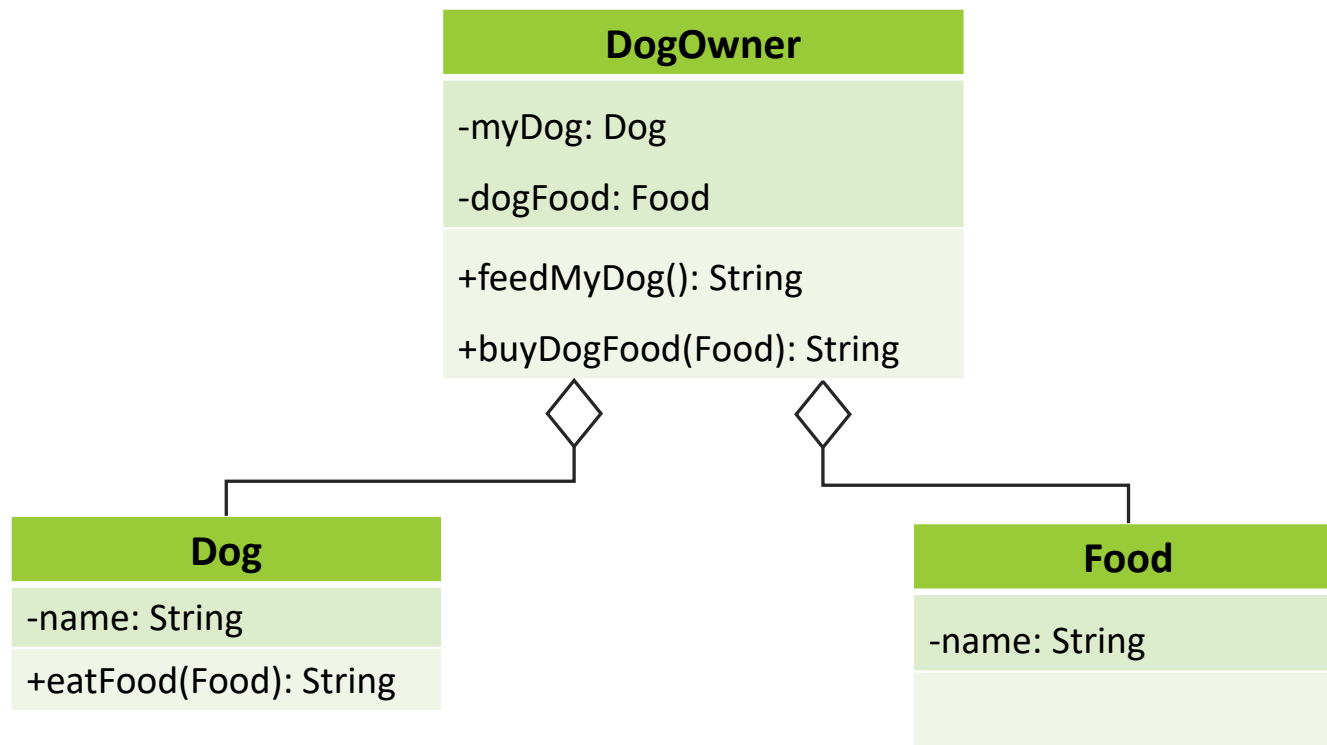
Separation of Concerns: Example

- Some basic behaviors that a dog can do are walking, running, speaking and eating
- Which behaviors can the dog do all on its own? And which ones need help from something or someone else?



Separation of Concerns: Example

- We need to separate two concerns, the action of eating and the action of providing food. This can be done by introducing a dog owner class



Conceptual Integrity

A series of horizontal lines in green and white, located at the bottom of the slide, extending from the left edge and ending on the right side.

Conceptual Integrity

- Conceptual integrity is about creating **consistent** software
- It is making decisions about how your system will be designed and implemented, so that even if multiple people worked on the software, it would seem as if there was only one mind guiding all the work
- **Fred Brooks**, a well-known computer architect, states in his book, *The Mythical Man-Month*:

“it is better to have a system omit certain anomalous features and improvements, but to reflect one set of design ideas, than to have one that contains many good but independent and uncoordinated ideas”

Conceptual Integrity

- One important way is communication. Adopting certain agile development practices like **daily stand-up meetings** and **sprint retrospectives**, where team members can agree to use certain libraries or methods when addressing certain issues, can help to maintain the consistency of the code. E.g., naming convention
- Another way is **code reviews**:
Systematic examinations of written code. It's often used to find mistakes in the software, but also to keep different developers consistent with each other
- Additionally, using certain design principles and programming constructs can also help in maintaining conceptual integrity. E.g.,:
 - **Java interfaces**: An interface defines a type, with behaviors that implementing classes of that type should all have in common.
 - **Design patterns**: They provide conventional structures for your classes to solve a design issue and lead to consistency

Conceptual Integrity

- Another approach is having a **well-defined architecture underlying your software** that describes how software, running as multiple processes, work together, and how they relate to each other
- **Unifying concepts** is also another approach to maintaining conceptual integrity. It is taking different things and finding common ground so that each concept can be seen and treated in similar ways. E.g.,:
 - In the Unix operating systems, every resource can be seen and manipulated as if it were a file
 - Another example is that in Java everything is an object

Inheritance Issues

A decorative graphic consisting of several horizontal lines in shades of green and white, located at the bottom of the slide.

Inheritance Issues

- Inheritance is a powerful design tool that can help you create clean, reusable, and maintainable software systems
- **Misusing inheritance can lead to poor code.** That happens when design principles are used improperly, creating more problems than they are meant to solve
- Is inheritance used to simply share attributes or behavior without further adding anything special in my subclasses? If the answer is yes, this is an indication of misuse, because there is no point for the subclasses to exist since the superclass already is enough

Inheritance Issues

```
public class Pizza {
    private List toppings;
    private String size;
    private String crustStyle;

    public Pizza(String size, String crust) {
        this.toppings = new ArrayList();
        this.size = size;
        this.crustStyle = crust;}

    public void addTopping(String topping) {
        this.toppings.add(topping);
    }

    public void bulkaddToppings(ArrayList toppingList) {
        this.toppings.addAll(toppingList);
    }

    public void cook() throw InterruptedException {
        wait(10 * 6000);
    }
}
```

Inheritance Issues

```
public class Pepperoni extends Pizza {  
    public Pepperoni(String size, String crust) {  
        super(size, crust);  
        super.addTopping("pepperoni");  
    }  
}
```

- There's no reason to use inheritance in this case, because you can simply use only the pizza class to build the pizza with pepperoni as a topping

Inheritance Issues: Liskov Substitution Principle

- The second indication of improper use of generalization is, if you break the **Liskov Substitution Principle**
- This principle states that a subclass can replace a superclass, if and only if, the subclass does not change the functionality of the superclass

Inheritance Issues

```
public class Animal {  
    private int numberOfLegs;  
    private boolean hasTail;  
  
    public Animal(int legs, boolean tail) {  
        this.numberOfLegs = legs;  
        this.hasTail = tail;  
    }  
  
    public void walk() { ... }  
    public void run() { ... }  
    public void eat() { ... }  
}
```

Inheritance Issues

```
public class Whale extends Animal {  
    public Whale() {  
        super(0, true);  
    }  
  
    private void swim() { ... }  
  
    public void run() {  
        this.swim();  
    }  
  
    public void walk() {  
        this.swim();  
    }  
}
```



Inheritance Issues: Stack class in Java

- An example of bad inheritance is the **stack class** in the Java Collections Library
- A stack is understood as *first in and last out* data structure. It has a small number of well-defined behaviors like peak, pop and push
- This is not the case in the Java stack class, because the stack class inherits from the **vector class!!**
- This means that the stack class can return an element at a specified index, retrieve the index of an element and even insert an element into a specific index

Modular design: Example

A decorative graphic element consisting of a solid green horizontal bar, followed by a white horizontal bar, and then several thin, parallel green horizontal lines of varying lengths extending to the right.

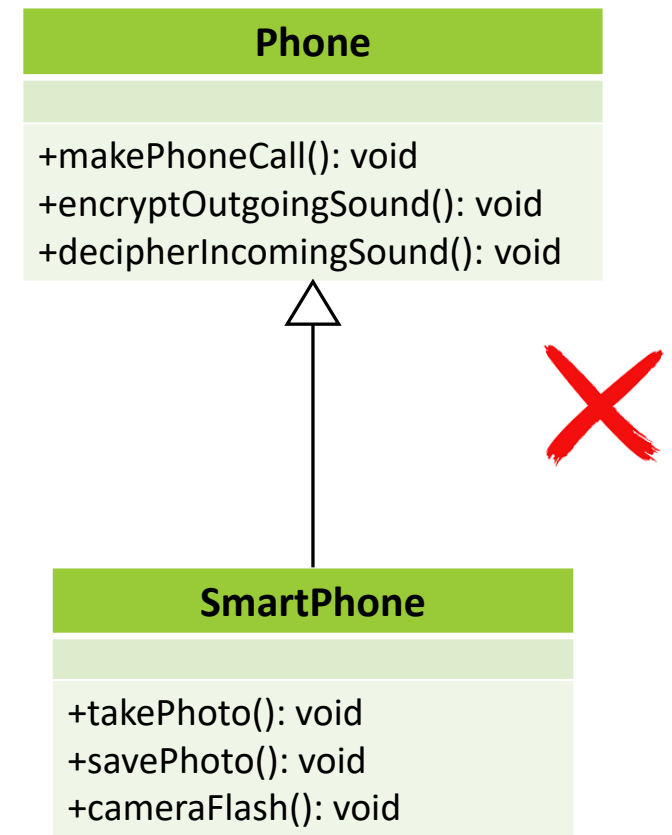
Modular design: Example

```
public class SmartPhone {  
    private byte camera;  
    private byte phone;  
  
    public SmartPhone() { ... }  
  
    public void takePhoto() { ... }  
    public void savePhoto() { ... }  
    public void cameraFlash() { ... }  
  
    public void makePhoneCall() { ... }  
    public void encryptOutgoingSound() { ... }  
    public void decipherIncomingSound() { ... }  
}
```

Low cohesion and no modularity!!

Modular design: Example

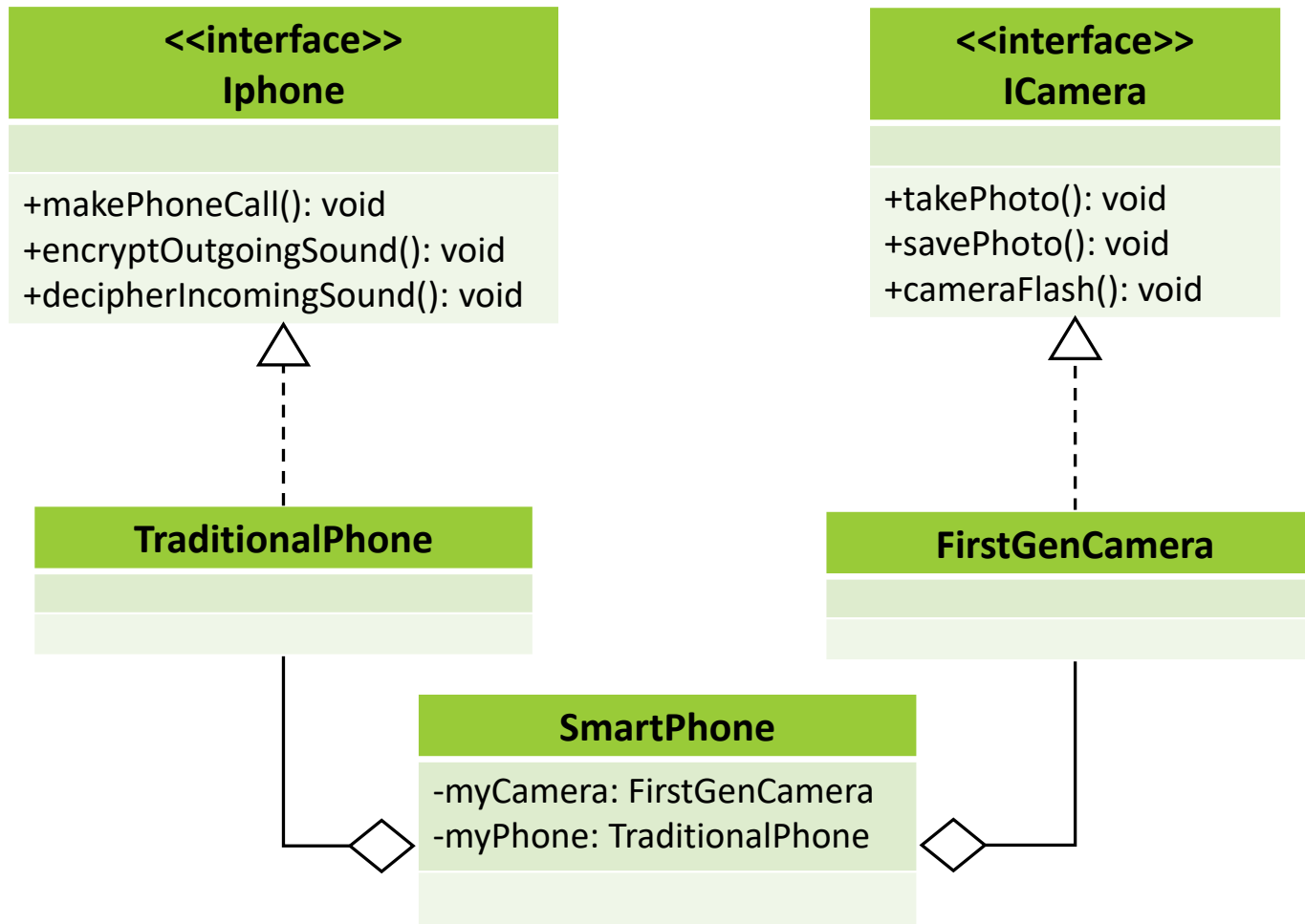
- Our SmartPhone class has two concerns
 - Act as a traditional telephone
 - Use built-in camera to take pictures



Modular design: Example

- What to change?
- We can separate the two concerns into their own more cohesive classes and encapsulate all the details about each into functionally distinct and independent classes
- The SmartPhone class will reference instances of our newly created classes, so that the smartphone can act as a coordinator of the camera and the phone
- This will let our smartphone provide access to all the behaviors of the camera and the phone without having to know how each component behaves

Modular design: Example



Modular design: Example

```
public interface ICamera {  
    public void takePhoto();  
    public void savePhoto();  
    public void cameraFlash();  
}
```

```
public interface IPhone {  
    public void makePhoneCall();  
    public void encryptOutgoingSound();  
    public void deciphereIncomingSound();  
}
```

```
public class FirstGenCamera implements ICamera { ... }
```

```
public class TraditionalPhone implements IPhone { ... }
```

Modular design: Example

```
public class SmartPhone {  
    private FirstGenCamera myCamera;  
    private TraditionalPhone myPhone;  
  
    public SmartPhone(FirstGenCamera aCamera,  
        TraditionalPhone aPhone ) {  
        this.myCamera = aCamera;  
        this.myPhone = aPhone;  
    }  
  
    public void useCamera() {  
        return this.myCamera.takePhoto();  
    }  
  
    public void usePhone() {  
        return this.myPhone.makePhoneCall();  
    }  
}
```

Modular design: Example

- If we want to swap out our camera or phone classes for something else, we do not need to touch any of the SmartPhone class's code
- The smartphone class is now more cohesive. The tradeoff is that we have increased coupling in our system because the smartphone class needs to know about the camera and phone interfaces and is indirectly dependent on other classes

Modular design: Example (Conclusion)

- Throughout this example, we use:
 - Separating out the general concerns of camera and phone through applying generalization and defining two interfaces
 - Separating out the functionality for a firstGenCamera and traditionalPhone by applying abstraction and encapsulation, and defining two implementing classes
 - Applying decomposition to the smartphone, so the parts are separated from the whole
- Deciding how to abstract, encapsulate, decompose and generalize to address the many concerns for a given problem is at the core of designing modular software

Model Checking



Model Checking

- Model checking is a **systematic check of your system's state model in all its possible states**
- Basically, you check all the various states of your software and find if there are any errors by simulating different events that would change the states and variables of your software
- So you check if the state model of your software contains some flaws. These checks are done using model checker software

Model Checking: Small world assumption

- The problem with model checking is the so-called **stage space explosion**. If I've got 1 million bits, I've 2 to the 1 million states. There's no way to explore that
- This means if the problem can't be described in a small number of bits, the model checker won't help
- Model checking relies on the small world assumption

Model Checking: Deadlock

- Model checking, agile development and test cases, they all fit together very nicely, because if you can specify what your system does, you can write test for it
- A rule that the software is required to satisfy is it must not produce a deadlock
- A **deadlock** is a situation where your system cannot continue because two tasks are waiting for the same resource
- Your model checker would simulate the different states that could occur in your system and if a deadlock is possible, it would provide details of the violation

Model Checking

- Using the information from the model checker, you advise yourself where to fix any problems, then run the checker again
- You do this until you can be sure that your software is correct with respect to the desired properties
- Model checking is a very useful **verification** technique. There are many ways to test your system's behavior, like unit testing, and simulations. But model checking can help find errors that these tests can't

References and Acknowledgments

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- Software Design and Architecture by Kenny Wong, University of Alberta Computing Science, Faculty of Science, Coursera
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- “Object-Oriented Patterns,” P. Coad; Comm. of the ACM, 9/92
- “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92

References and Acknowledgments

- The Art of Agile Development 1st Edition, by James Shore and Shane Warden
- Essential Scrum: A Practical Guide to the Most Popular Agile Process, Kenneth S. Rubin's Essential Scrum, 2012
- Scrum: The Art of Doing Twice the Work in Half the Time, Jeff Sutherland, 2014
- The Scrum Field Guide: Practical Advice For Your First year, Mitch Lacey, 2012
- Agile Retrospectives: Making Good Teams Great by Esther Derby & Diana Larsen
- User Story Mapping: Discover the Whole Story, Build the Right Product, 2021
- The Deadline: A Novel about Project Management
- Agile Leadership Toolkit by Peter Koning, 2019
- Akhtar, A., Bakhtawar, B. and Akhtar, S., 2022. EXTREME PROGRAMMING VS SCRUM: A COMPARISON OF AGILE MODELS. International Journal of Technology, Innovation and Management (IJTIM), 2(2).
- Ashari, I.F., Aryani, A.J. and Ardhi, A.M., 2022. Design and Build Inventory Management Information System Using the Scrum Method. JSil (Jurnal Sistem Informasi), 9(1), pp.27-35.

Images Acknowledgments

- **Puzzle pieces.** by velkr0, used under CC. BY 2.0. Image cropped, brightness/contrast adjusted.
<https://www.flickr.com/photos/velkr0/6573677629/>
- **LEGO blocks (#0047).** by regan76, used under CC BY 2.0. Image cropped, brightness/ contrast adjusted.
https://www.flickr.com/photos/j_regan/6454392293/

End of Lecture